Pixel Pattern Classifier

Kim, Justin
Student at Bellevue College
justinkimkim.kim@bellevuecollege.edu

Ly, Hengmeng
Student at Bellevue College
hengmeng.ly@bellevuecollege.edu

Ho, Bella
Student at Bellevue College
bella.ho@bellevuecollege.edu

***Abstract*—This paper presents the design, implementation, and analysis of a pixel pattern classifier that assigns structural and categorical classifications to pixels within an image without the use of machine learning libraries. Images are backgrounds with filled pixels (represented as 0s and 1s). The system transforms symbols from raw 2D grids into normalized 16x16 feature vectors. Classification is achieved by assessing Cosine Similarity, Euclidean Distance, and descriptive structural tiebreakers such as symmetry and center of mass distributions.**

Keywords—pixel pattern classifier, feature normalization, cosine similarity, structural features, pattern recognition.

I. Introduction

By accessing geometric patterns like structural edges, textures, and local color intensities, the pixel pattern classifier can assign a class/category to every individual pixel in an image. However, the pixel pattern classifier stated in this report is designed to isolate background components from filled pixels, represented as 0s and 1s.

The core of the project takes a variable 2-dimensional (2D) grid of binary pixels representing a hand drawn work, processes it through normalization functions, and verifies its class against a registered library of known ground-truth templates. While the fundamental math mirrors machine learning models, this project does not incorporate any extern machine learning libraries, opting to operate through algorithms.

II. Classification Process

A. Step 1: The Bounding Box

The raw input grid can be any size up to a max of 32x32 and often has empty background whitespace. The *cropToBoundingBox* function scans all four edges of the grid inward to find the first and last row and column containing a filled pixel. This produces a BoundingBox struct with four values: minRow, maxRow, minCol, and maxCol.

To clarify, minRow starts at the maximum possible value (rows), and maxRow starts at –1, so the very first filled pixel found always beats both.

B. Step 2: Normalizing to 16x16

Following the bounding box, the symbol is scaled to a fixed 16x16 grid regardless of its original size. For each of the 256 output cells, the corresponding source is computed by mapping the output position back to the bounding box proportionally.

The formula used is: $newRow = box.minRow + \lfloor r \times Height / Target \rfloor$

Height is the bounding box's height with a +1 correction, and a min() is used to prevent index overflow at the edges. The result is then stored as a flat 1D array of 256 doubles.

The conversion is: $Index = r \times 16 + c$

The flattening is used to turn every symbol into a point to make comparisons purely mathematical.

C. Step 3: Template Library

To identify incoming shapes, the program relies on a reference dictionary. Template structs are created, each containing a **label char[32]**, a standard feature array **double[256]**, and computed structural features.

Currently, up to 20 templates are available for storage.

Templates are added via the addTemplate() function, which runs the symbol through steps 1 and 2, then stores the result. Structural features, which will be clarified soon, are extracted once at the time of registry rather than computed during every classification.

D. Step 4: Vector Similarity and Structural Features

Each input grid is compared against every template using two metrics: Cosine Similarity [1] and Euclidean Distance [2].

$$Similarity = (A \cdot B) / (||A|| \times ||B||)$$

An output of 1.0 represents perfect alignment, whereas 0.0 means the vectors are perpendicular.

Euclidean Distance refers to the raw point to point pixel difference:

$$Distance = \sqrt{\sum}(Ai - Bi)^2$$

A separation of 0 represents a complete match. Lower values are more accurate, so the metric is converted to a 0-1 score via 1.0/(1.0 + distance) before being weighed.

Finally, Structural Features provide more information and act as tiebreakers with the least weight:

- **Horizontal Symmetry** (compares left half against mirrored right half)
- **Vertical Symmetry** (compares the top half against a mirrored bottom half)
- **Pixel Density** (fraction of filled pixels)
- **Center of Mass** (weighted average position of filled pixels)

E. Step 5: Confidence Scoring and Classification

All scores from the previous steps are merged into a final confidence metric using a weighted formula:

Confidence = (CosSim × 0.5) + (DistScore × 0.2) + (SymH × 0.1) + (SymV × 0.1) + (Density × 0.1)

Cosine similarity is weighted at the highest (50%), because it is the most direct pixel comparison. Structural Features are restricted to a combined total of 30% to serve as tiebreakers. In addition, to prevent false positives, a strict confidence threshold is in place: if the best confidence score fails to reach 0.7 (70%) threshold, the classification is rejected.

III. Data Structures

Structure	Type	Use
grid[][]	int[32][32]	Raw pixel input (0 is background, 1 is filled)
outputFeatures[]	double[256]	Flattened 16x16
Template[]	struct array[20]	Ground truth dictionary of known symbols
BoundingBox	Struct	Four crop edges from Step 1
StructuralFeatures	Struct	Symmetry, density, and center of mass
ClassifyResult	Struct	Label, confidence, runner-up, and rejection

IV. Challenges

During intial development, several complications occurred. Struct initialization order in the header file requires strict layout restrictions. Because the **Template** struct relies on an embedded **StructuralFeatures** member, the latter must be declared first. Having **Template** before **StructuralFeatures** caused compiler errors, so the structs had to be reconfigured.

Furthermore, visual cues from the browser version of IntelliSense intiallty flagged several lines as containing errors. Specifically, around the structure of member arrays to functions. The actual compiler, MSVC, had no issues with the same code.

The Bounding Box Height also had an initial error. The original resizing formula computed **height** as *(maxRow – minRow)* without the +1. Meaning, a symbol of rows 2-4 would compute a **height** of 2 rather than 3. Adding +1 to both **height** and **width** variables fixed the scaling.

An early version of the distance function contained an array **features2[1]**, instead of **features2[i]** inside the loop. Finally, the **distance score**, used in computing confidence, was written as 1/(1+distance) using integers. Since both 1 values were an integer, C++ truncated the result which is horribly inaccurate when distance must be from 0.0-1.0.

V. Experimental Tests

The system was verified through clean, noisy, and random input variants to measure the sensitivity and performance. Clean baseline symbols had maximum confidence marks, which is to be expected, whereas noisy inputs had lower confidence scores.

Test Input	Prediction	Confidence	Correct?
Clean X	X	100%	Yes
Clean O	O	100%	Yes
Clean T	T	100%	Yes
Noisy X	X	73.00%	Yes
Very Noisy X	T (Rejected)	63.33% (Below Threshold)	No
Noisy O	O	70.69%	Yes
Noisy T	T	71.79%	Yes
Large X	X	55.97% (Below)	Yes
Random	O	61.97% (Below)	No (random)

VI. Reflections

Raising the confidence threshold from 0.5 to 0.7 was necessary to balance precision. The classifier now rejects heavily inaccurate inputs rather than guessing wrong, which is reasonable since a wrong label is worse than no label. In a different context where rejection is undesirable (categorizing larger groups clumped together), a lower threshold might be preferable.

Furthermore, structural features for templates are computed once during addTemplate() and stored inside the Template struct. An earlier design that recomputed them on every classify() call was used, but since the template features never change, computing them once is incredibly more efficient.

VII. Conclusion

The Pixel Pattern Classifier successfully identifies clean symbols, handles moderate noise, and rejects heavily edited or random inputs. The base revolves around math, specifically cosine similarity in a 256-dimensional feature space with structural feature checks. The project mirrors computer vision systems, albeit at an extremely rudimentary level, demonstrates that pattern recognition does not require machine learning libraries.

VIII. References

[1] scikit-learn developers. (2026). 8.8. Pairwise metrics, Affinities and Kernels — Cosine similarity. scikit-learn 1.9 documentation. Retrieved from https://scikitlearn.org/stable/modules/metrics.html#cosine-similarity

[2] GeeksforGeeks. (2025). Euclidean Distance | Formula, Derivation & Solved Examples. Retrieved from https://www.geeksforgeeks.org/maths/euclidean-distance/